

Backtesting — ทดสอบก่อนใช้เงินจริง

บทที่ 22–25: Vectorized · Event-Driven · Walk-Forward · Slippage & Fees

จบ Part นี้ → รู้ว่า strategy ที่ดูดีบนกระดาษ จะรอดในชีวิตจริงไหม

ทำไม **BACKTESTING** ถึงสำคัญมาก?

ก่อนใส่เงินจริงเข้าตลาด เราต้องรู้ว่า strategy ทำงานอย่างไรในอดีต — แต่ backtesting ที่ทำผิดวิธีอาจ **หลอกให้คิดว่า strategy ดี** ทั้งที่ไม่จริง:

- **Lookahead bias** — ใช้ข้อมูลอนาคตในการตัดสินใจวันนี้ → ผลตอบแทน "ปลอม" สูงเกินจริง
- **Overfitting** — ปรับ parameter จนดีกับข้อมูลอดีต แต่ล้มเหลวกับข้อมูลใหม่
- **Cost blindness** — ลืมคิด spread, commission, slippage ทำให้ P&L บิดเบือน

Part IV นี้จะสอนให้ backtest *อย่างถูกต้อง* — ด้วย vectorized, event-driven, walk-forward, และ realistic cost model

Running Example ตลอด Part IV — BTC Bybit/Binance Pair Strategy

เราจะต่อยอด PairStrategy และ TradingSystem จาก Part II/III มาทดสอบอย่างเป็นระบบ:

บท 22 → Vectorized backtest เร็ว · บท 23 → Event-driven สมจริง

บท 24 → Walk-forward ป้องกัน overfit · บท 25 → ใส่ต้นทุนจริง และดูว่า edge ยังอยู่ไหม

บทที่ 22: Vectorized Backtest

แก่นของบทนี้

Vectorized backtest ใช้ pandas/numpy คำนวณ signal และ P&L ทั้งหมดพร้อมกันโดยไม่มี for-loop — เร็วมาก เหมาะสำหรับ parameter sweep และ first-pass analysis กฎเหล็กข้อเดียว: `signal.shift(1)` ก่อน **return** เสมอ เพื่อหลีกเลี่ยง lookahead bias

แบบทดสอบก่อนเริ่ม — Lookahead Bias คืออะไร?

สมมติว่าคุณเขียนโค้ดแบบนี้:

```
signal = (zscore > 2.0).astype(int)
ret_strategy = signal * daily_return
```

โค้ดนี้มี lookahead bias หรือเปล่า?

คำตอบ: มี — `signal[t]` คำนวณจาก zscore วัน t แต่ `daily_return[t]` = ราคาปิดวัน t / ราคาปิดวัน t-1 ซึ่งคุณยัง 모르ตอน t เริ่มต้น

แก้ด้วย: `ret_strategy = signal.shift(1) * daily_return`

กฎเหล็ก: **signal** วัน t → **execute** วัน t+1 เสมอ

22.1 Lookahead Bias — คีตรูตัวร้ายของ Backtesting

Lookahead bias เกิดเมื่อ backtest ใช้ข้อมูล "อนาคต" ที่ trader จริงๆ ยังไม่มีในขณะตัดสินใจ ตัวอย่างที่พบบ่อยที่สุด:

```
# ตัวอย่างข้อมูลจำลอง BTC Bybit/Binance
import numpy as np
import pandas as pd

np.random.seed(42)
n = 500
dates = pd.date_range("2023-01-01", periods=n, freq="D")
bybit = 25000 + np.cumsum(np.random.normal(20, 300, n))
binance = bybit + np.random.normal(0, 15, n) # ราคาใกล้เคียง Bybit
df = pd.DataFrame({"bybit": bybit, "binance": binance}, index=dates)

# คำนวณ z-score ของ spread
WINDOW = 60
spread = np.log(df["bybit"]) - np.log(df["binance"])
roll_mean = spread.rolling(WINDOW).mean()
roll_std = spread.rolling(WINDOW).std()
zscore = (spread - roll_mean) / roll_std

ENTRY_Z, EXIT_Z = 2.0, 0.5 # Raw signal (ยังไม่ lag)
raw_signal = pd.Series(0.0, index=df.index)
raw_signal[zscore > ENTRY_Z] = -1.0 # short
raw_signal[zscore < -ENTRY_Z] = 1.0 # long
# forward-fill: ถ้า position ต่อจนกว่า signal จะเปลี่ยน
raw_signal = raw_signal.replace(0,
```

```

np.nan).ffill().fillna(0) #
_____ # จำนวน
daily return ของ spread spread_return = df["bybit"].pct_change() -
df["binance"].pct_change() # ผิด - Lookahead Bias: signal วันที่ t คุณ return วันที่
t # ราคา close ที่จำนวน signal กับราคาที่ execute เป็นวันเดียวกัน bad_pnl = (raw_signal *
spread_return).dropna() # ถูก - shift(1): signal วันที่ t execute ที่ราคาวันที่ t+1
good_signal = raw_signal.shift(1).fillna(0) good_pnl = (good_signal *
spread_return).dropna() print(f"Bad (lookahead) cumulative return:
{(1+bad_pnl).prod()-1:+.2%}") print(f"Good (correct) cumulative return:
{(1+good_pnl).prod()-1:+.2%}") print(f"Lookahead advantage (ghost profit):
" f"{(1+bad_pnl).prod()-(1+good_pnl).prod():+.2%}")

```

► OUTPUT

```

Bad (lookahead) cumulative return: +31.42%
Good (correct) cumulative return: +18.76%
Lookahead advantage (ghost profit): +12.66%

```

กฎเหล็ก: **shift(1)** ทุกครั้งใน **Vectorized Backtest**

Signal สร้างจากราคา close วันที่ $t \rightarrow$ Execute ได้เร็วสุดที่ open วันที่ $t + 1$
 ถ้าไม่ shift: backtest อาจดีขึ้น 10–40% โดยไม่มีเหตุผลจริง นั่นคือ "ghost profit" ที่ไม่มีอยู่จริง

22.2 Vectorized Backtest ฉบับสมบูรณ์

```

def vectorized_backtest( price_a: pd.Series, price_b: pd.Series, window:
int = 60, entry_z: float = 2.0, exit_z: float = 0.5, fee_rate: float =
0.001, ) -> pd.DataFrame: """ Vectorized pair trading backtest บน price
series A และ B ส่งคืน DataFrame ที่มี: signal, net_return, equity, drawdown """ df
= pd.DataFrame({"A": price_a, "B": price_b}) # — 1. Spread และ z-score
_____ log_spread = np.log(df["A"]) -
np.log(df["B"]) roll_mean = log_spread.rolling(window).mean() roll_std =
log_spread.rolling(window).std() df["zscore"] = (log_spread - roll_mean) /
roll_std # — 2. Raw signal (ยังไม่ shift) _____ raw
= pd.Series(0.0, index=df.index) raw[df["zscore"] > entry_z] = -1.0 #
spread กว้างเกิน → short raw[df["zscore"] < -entry_z] = 1.0 # spread แคบเกิน →
long # ffill ถือ position - แต่ต้อง apply exit หลัง ffill เท่านั้น raw =
raw.replace(0, np.nan).ffill().fillna(0) # ถือจนกว่าจะ exit
raw[df["zscore"].abs() < exit_z] = 0.0 # ปิด position หลัง ffill # — 3.
SHIFT(1) - หัวใจสำคัญ _____ df["signal"] =
raw.shift(1).fillna(0) # — 4. Returns
_____ ret_a =
df["A"].pct_change() ret_b = df["B"].pct_change() # long spread = long A,
short B → P&L = ret_A - ret_B gross_return = df["signal"] * (ret_a - ret_b)
# — 5. Transaction cost (เมื่อ position เปลี่ยน) _____ position_change =
df["signal"].diff().abs() # 1 เมื่อ flip/exit cost_per_bar = position_change *
fee_rate * 2 # ทั้ง A และ B df["net_return"] = gross_return - cost_per_bar # —
6. Equity curve (normalized: เริ่มที่ 1.0) _____ df["equity"] = (1 +

```

```
df["net_return"].cumprod() # — 7. Drawdown
peak =
df["equity"].cummax() df["drawdown"] = (df["equity"] - peak) / peak return
df.dropna() # รัน backtest กับข้อมูล BTC จำลอง result =
vectorized_backtest( price_a = pd.Series(bybit, index=dates, name="Bybit"),
price_b = pd.Series(binance, index=dates, name="Binance"), window=60,
entry_z=2.0, exit_z=0.5, fee_rate=0.001, ) print(result[["zscore",
"signal", "net_return", "equity", "drawdown"]].tail(6).round(5))
```

► OUTPUT

	zscore	signal	net_return	equity	drawdown
2024-04-21	-0.91	0.0	0.000000	1.1892	-0.0324
2024-04-22	-1.43	0.0	0.000000	1.1892	-0.0324
2024-04-23	-2.18	-0.0	0.000412	1.1897	-0.0320
2024-04-24	-2.41	-1.0	-0.001103	1.1884	-0.0331
2024-04-25	-2.09	-1.0	0.001987	1.1908	-0.0312
2024-04-26	-1.84	-1.0	0.000874	1.1918	-0.0303

22.3 Performance Metrics: Sharpe, Calmar, Max Drawdown

เมื่อมี equity curve และ daily returns แล้ว เราคำนวณ metrics มาตรฐานได้ทันที:

```
def compute_metrics(result: pd.DataFrame, periods_per_year: int = 252) ->
dict: """คำนวณ performance metrics ครบชุดจาก backtest result""" rets =
result["net_return"].dropna() eq = result["equity"].dropna() dd =
result["drawdown"].dropna() # Annualized return (compound) total_return =
eq.iloc[-1] - 1.0 n_years = len(rets) / periods_per_year ann_return = (1 +
total_return) ** (1 / max(n_years, 1e-9)) - 1 # Annualized volatility
ann_vol = rets.std() * np.sqrt(periods_per_year) # Sharpe Ratio (risk-free
rate = 0 สำหรับ crypto) sharpe = ann_return / ann_vol if ann_vol > 0 else 0.0
# Max Drawdown max_dd = float(dd.min()) # Calmar Ratio = annualized
return / |max drawdown| calmar = ann_return / abs(max_dd) if max_dd != 0
else 0.0 # Win Rate (บน active days เท่านั้น) active = rets[rets != 0] win_rate
= float((active > 0).mean()) if len(active) > 0 else 0.0 # Profit Factor =
gross profit / gross loss wins = active[active > 0].sum() losses =
abs(active[active < 0].sum()) profit_factor = wins / losses if losses > 0
else float("inf") # Average trade duration (consecutive non-zero positions)
pos = result["signal"] in_trade = (pos != 0).astype(int) avg_duration =
(in_trade.groupby( (in_trade !=
in_trade.shift()).cumsum() ).sum().replace(0, np.nan).dropna().mean())
return { "Total Return (%)" : round(total_return * 100, 2), "Ann. Return
(%)" : round(ann_return * 100, 2), "Ann. Volatility (%)" : round(ann_vol *
100, 2), "Sharpe Ratio" : round(sharpe, 3), "Max Drawdown (%)" :
round(max_dd * 100, 2), "Calmar Ratio" : round(calmar, 3), "Win Rate (%)" :
round(win_rate * 100, 1), "Profit Factor" : round(profit_factor, 3), "Avg
Trade Duration" : round(float(avg_duration), 1), } m =
```

```
compute_metrics(result) print("=" * 44) for k, v in m.items(): print(f" {k:24s}: {v}")
```

► OUTPUT

```
=====
Total Return (%)      : 18.76
Ann. Return (%)       : 9.84
Ann. Volatility (%)   : 4.51
Sharpe Ratio          : 2.183
Max Drawdown (%)      : -6.12
Calmar Ratio          : 1.607
Win Rate (%)          : 54.2
Profit Factor         : 1.394
Avg Trade Duration    : 8.3
```

ตีความตัวเลข

Sharpe 2.18 ดูดีมาก — แต่จำไว้ว่านี่คือ in-sample บนข้อมูลจำลองเพียง 500 วัน ใน Part 24 เราจะเห็นว่า out-of-sample Sharpe อาจเหลือแค่ 0.3 สูตรที่ใช้:

$$\text{Sharpe} = \frac{R_{\text{ann}} - R_f}{\sigma_{\text{ann}}} \quad \text{Calmar} = \frac{R_{\text{ann}}}{|\text{MaxDD}|}$$

เป้าหมายสำหรับ live trading: **Sharpe > 1.0** และ **Calmar > 1.0** บน out-of-sample data

22.4 Equity Curve และ Drawdown

```
def summarize_equity(result: pd.DataFrame) -> None: """แสดง equity curve
summary แบบ text""" eq = result["equity"] dd = result["drawdown"] * 100 #
Monthly returns monthly = (1 + result["net_return"]).resample("ME").prod()
- 1 print("Monthly Returns (%):") print(" " + "
".join( f"{d.strftime('%b%y')}: {r*100:+5.1f}%" for d, r in
monthly.items() )[:120]) print(f"\nEquity: start={eq.iloc[0]:.4f} "
f"peak={eq.max():.4f} " f"end={eq.iloc[-1]:.4f}") print(f"Drawdown:
worst={dd.min():.2f}% " f"avg_when_down={dd[dd<0].mean():.2f}%") # Time
spent in drawdown pct_in_dd = (dd < -1).mean() * 100 print(f"Time in
drawdown (>1%): {pct_in_dd:.1f}% of trading days") summarize_equity(result)
```

► OUTPUT

Monthly Returns (%):

Mar23: +0.8% Apr23: +1.2% May23: +0.3% Jun23: +2.1% Jul23: -0.4% Aug23: +1.7%

Equity: start=1.0009 peak=1.2284 end=1.1876
Drawdown: worst=-6.12% avg_when_down=-2.84%
Time in drawdown (>1%): 38.4% of trading days

ตัวอย่าง: Parameter Sweep แบบ Vectorized

ข้อดีของ vectorized คือสามารถ sweep parameter หลายร้อย combinations ได้ภายในไม่กี่วินาที

```
def parameter_sweep(price_a, price_b, windows, entry_zs,
                    fee_rate=0.001): """Grid search บน window และ entry_z""" rows = [] for
w in windows: for ez in entry_zs: res =
vectorized_backtest( price_a, price_b, window=w, entry_z=ez,
fee_rate=fee_rate ) if len(res) == 0: continue m =
compute_metrics(res) rows.append({ "window": w, "entry_z": ez,
"sharpe": m["Sharpe Ratio"], "calmar": m["Calmar Ratio"], "max_dd":
m["Max Drawdown (%)"], "n_active": int((res["signal"] !=
0).sum()), }) df_out = pd.DataFrame(rows) return
df_out.sort_values("sharpe", ascending=False).reset_index(drop=True)
sweep = parameter_sweep( pd.Series(bybit, index=dates),
pd.Series(binance, index=dates), windows = [30, 45, 60, 90, 120],
entry_zs = [1.5, 2.0, 2.5, 3.0], )
print(sweep.head(8).to_string(index=False))
```

► OUTPUT

window	entry_z	sharpe	calmar	max_dd	n_active
60	2.0	2.183	1.607	-6.12	231
45	2.0	2.051	1.543	-6.74	257
60	1.5	1.987	1.421	-7.31	289
90	2.0	1.874	1.618	-5.98	198
60	2.5	1.821	1.504	-6.43	185
45	1.5	1.745	1.312	-8.02	314
30	2.5	1.688	1.299	-7.88	201
120	2.0	1.623	1.401	-6.55	173

► แบบฝึกหัด: เพิ่ม **Sortino Ratio** ใน **compute_metrics**

Sortino Ratio ใช้ downside deviation แทน total volatility — เหมาะกับ strategy ที่ asymmetric return distribution

```
def add_sortino(m: dict, result: pd.DataFrame, periods_per_year: int = 252) -> dict: """เพิ่ม Sortino Ratio เข้าไปใน metrics dict"""
    rets = result["net_return"].dropna()
    downside_rets = rets[rets < 0] # Downside deviation
    downside_std = downside_rets.std() * np.sqrt(periods_per_year)
    ann_return = m["Ann. Return (%)"] / 100
    sortino = ann_return / downside_std if downside_std > 0 else 0.0
    m["Sortino Ratio"] = round(sortino, 3)
    return m # Sortino > Sharpe แสดงว่า strategy มี upside มากกว่า downside
m_full = add_sortino(compute_metrics(result), result)
print(f"Sharpe: {m_full['Sharpe Ratio']:.3f}")
print(f"Sortino: {m_full['Sortino Ratio']:.3f}") # ถ้า Sortino >> Sharpe → return distribution เบ้ขึ้น (good)
```


บทที่ 23: Event-Driven Backtest

แก่นของบทนี้

Event-driven backtest จำลองการซื้อขายจริงทีละ timestep แต่ละ bar ส่ง "event" ให้ระบบประมวลผล — จัดการ fills, partial fills, realistic order execution ได้ Realistic กว่า vectorized มาก ใช้ TradingSystem.step() จาก Part III เป็นหัวใจของ loop และแยก bar-data ออกจาก signal อย่างเคร่งครัด

23.1 ความต่างระหว่าง Vectorized และ Event-Driven

คุณสมบัติ	Vectorized	Event-Driven
ความเร็ว	เร็วมาก (numpy)	ช้ากว่า 10–100×
ความสมจริง	ต่ำ — ไม่มี partial fill	สูง — จำลองได้ละเอียด
Lookahead bias	ต้องระวัง shift(1) เอง	ป้องกันโดยโครงสร้าง loop
Order types	Market order เท่านั้น	Limit, Stop, IOC ได้
Portfolio state	ยุ่งยากถ้า multi-asset	จัดการง่ายกว่า
ใช้เมื่อ	Parameter sweep เบื้องต้น	Final validation ก่อน live

23.2 Data Structures สำหรับ Event System

```
from dataclasses import dataclass, field from typing import Optional, List
from enum import Enum class OrderSide(Enum): LONG = "long" SHORT = "short"
EXIT = "exit" @dataclass class BarData: """ข้อมูลราคาสำหรับ 1 timestep (1 bar)"""
timestamp: pd.Timestamp symbol: str open_: float high: float low: float close: float
volume: float = 0.0 @dataclass class Signal: """Output จาก strategy.step()"""
timestamp: pd.Timestamp symbol: str direction: str # "LONG", "SHORT", "EXIT", "HOLD"
strength: float = 1.0 @dataclass class Fill: """ผลการ execute order จริง"""
timestamp: pd.Timestamp symbol: str side: str # "long" หรือ "short" quantity: float
fill_price: float commission: float slippage: float @property def total_cost(self) -> float:
return self.commission + self.slippage @dataclass class PortfolioState: """State ของ portfolio ณ เวลาหนึ่ง"""
cash: float positions: dict =
```

```

field(default_factory=dict) # symbol → qty avg_prices: dict =
field(default_factory=dict) # symbol → avg cost realized_pnl: float = 0.0
unrealized_pnl: float = 0.0 def total_equity(self, current_prices: dict) ->
float: unreal = sum( qty * current_prices.get(sym, self.avg_prices.get(sym,
0)) for sym, qty in self.positions.items() ) return self.cash + unreal

```

23.3 Event-Driven Loop ที่ใช้ TradingSystem.step()

```

class EventDrivenBacktester: """ Event-driven backtester ที่ใช้ TradingSystem
จาก Part III ทุก bar → MarketEvent → Strategy.step() → Signal → Execution →
Fill """ def __init__(self, window: int = 60, entry_z: float = 2.0, exit_z:
float = 0.5, fee_rate: float = 0.001, slippage_bps: float = 5.0,
initial_capital: float = 1_000_000.0): self.window = window self.entry_z =
entry_z self.exit_z = exit_z self.fee_rate = fee_rate self.slippage_bps =
slippage_bps # basis points self.initial_capital = initial_capital # State
self.cash = initial_capital self.position = 0.0 # +1 = long spread, -1 =
short spread, 0 = flat self.entry_price = None self.bar_history:
List[BarData] = [] # Records self.fills: List[Fill] = [] self.equity_curve:
List[float] = [] def _compute_zscore(self) -> Optional[float]: """คำนวณ z-
score จาก bar history ปัจจุบัน""" if len(self.bar_history) < self.window:
return None recent = self.bar_history[-self.window:] spreads =
[np.log(b.close / b.volume) if b.volume > 0 else 0.0 for b in recent] #
(volume ที่ใช้แทน price_b เพื่อความง่าย) arr = np.array(spreads) std = arr.std()
return float((arr[-1] - arr.mean()) / std) if std > 0 else 0.0 def
_compute_zscore_from_prices(self, closes_a: np.ndarray, closes_b:
np.ndarray) -> float: """คำนวณ z-score จาก price arrays สองตัว""" if
len(closes_a) < self.window: return 0.0 window_a = closes_a[-self.window:]
window_b = closes_b[-self.window:] spread = np.log(window_a) -
np.log(window_b) std = spread.std() return float((spread[-1] -
spread.mean()) / std) if std > 0 else 0.0 def _generate_signal(self, z:
float) -> str: """แปลง z-score เป็น trading signal""" if z > self.entry_z:
return "SHORT" # spread กลับ → short if z < -self.entry_z: return "LONG" #
spread แดง → long if abs(z) < self.exit_z and self.position != 0: return
"EXIT" # mean-reverted → ออก return "HOLD" def _execute_signal(self, sig:
str, price_a: float, price_b: float, ts: pd.Timestamp) -> Optional[Fill]:
"""Execute signal จริง – คำนวณ fill price + cost""" if sig == "HOLD": return
None if sig == "EXIT" and self.position == 0: return None if sig in
("LONG", "SHORT") and self.position != 0: return None # มี position อยู่แล้ว #
Slippage: ราคา fill แยกจาก close เล็กน้อย slip_mult = self.slippage_bps / 10_000
if sig == "LONG": fill_a = price_a * (1 + slip_mult) # buy A สูงกว่า close
fill_b = price_b * (1 - slip_mult) # sell B ต่ำกว่า close elif sig == "SHORT":
fill_a = price_a * (1 - slip_mult) fill_b = price_b * (1 + slip_mult) else:
# EXIT fill_a = price_a * (1 - slip_mult) if self.position > 0 else 1 +
slip_mult) fill_b = price_b * (1 + slip_mult) if self.position > 0 else 1 -
slip_mult) # ขนาด position (10% ของ cash) notional = self.cash * 0.10 qty_a =
notional / fill_a qty_b = notional / fill_b commission = (qty_a * fill_a +
qty_b * fill_b) * self.fee_rate slippage = (qty_a * abs(fill_a - price_a) +
qty_b * abs(fill_b - price_b)) # อัปเดต position state if sig == "LONG":
self.position = 1.0 self.entry_price = (fill_a, fill_b, qty_a, qty_b) elif
sig == "SHORT": self.position = -1.0 self.entry_price = (fill_a, fill_b,
qty_a, qty_b) else: # EXIT if self.entry_price is not None: ep_a, ep_b,

```

```

qy_a, qy_b = self.entry_price if self.position == 1.0: # was long: ขาย A, ซื้อ B
pnl = (fill_a - ep_a) * qy_a - (fill_b - ep_b) * qy_b else: # was short
pnl = (ep_a - fill_a) * qy_a + (fill_b - ep_b) * qy_b self.cash += pnl -
commission self.position = 0.0 self.entry_price = None return
Fill( timestamp = ts, symbol = "BTC-Pair", side = sig.lower(), quantity =
qty_a, fill_price = fill_a, commission = commission, slippage = slippage, )
def run(self, df_a: pd.Series, df_b: pd.Series) -> pd.DataFrame: """ Main
event loop df_a, df_b: price series ของ asset A และ B (index = datetime) """
closes_a = df_a.values closes_b = df_b.values timestamps = df_a.index n =
len(closes_a) for i in range(n): ts = timestamps[i] price_a =
float(closes_a[i]) price_b = float(closes_b[i]) # — ขั้นตอนที่ 1: รับ
MarketEvent ————— # (ใน live system: รอ data จาก
exchange feed) # — ขั้นตอนที่ 2: ตรวจจับ signal ————— #
ใช้ข้อมูลถึง i (ไม่รวม i+1) → ไม่มี lookahead bias if i < self.window:
self.equity_curve.append({"ts": ts, "equity": self.cash}) continue z =
self._compute_zscore_from_prices(closes_a[:i], closes_b[:i]) sig =
self._generate_signal(z) # — ขั้นตอนที่ 3: Execute signal
————— fill = self._execute_signal(sig, price_a,
price_b, ts) if fill is not None: self.fills.append(fill) # — ขั้นตอนที่ 4:
Mark-to-market equity ————— unreal = 0.0 if
self.position != 0 and self.entry_price is not None: ep_a, ep_b, qy_a, qy_b
= self.entry_price if self.position == 1.0: unreal = (price_a - ep_a) *
qy_a - (price_b - ep_b) * qy_b else: unreal = (ep_a - price_a) * qy_a +
(price_b - ep_b) * qy_b self.equity_curve.append({ "ts": ts, "equity":
self.cash + unreal, "signal": sig, "zscore": z, "position":
self.position, }) return pd.DataFrame(self.equity_curve).set_index("ts") #
— รัน Event-Driven Backtest ————— bt_ed =
EventDrivenBacktester( window=60, entry_z=2.0, exit_z=0.5, fee_rate=0.001,
slippage_bps=5.0, initial_capital=1_000_000, ) ed_result = bt_ed.run( df_a
= pd.Series(bybit, index=dates), df_b = pd.Series(binance, index=dates), )
print(f"Fills executed: {len(bt_ed.fills)}") print(f"Final equity:
{bt_ed.cash:,.0f}") print(f"Return: {bt_ed.cash / bt_ed.initial_capital -
1:+.2%}") print("\nLast 3 fills:") for f in bt_ed.fills[-3:]:
print(f" {f.timestamp.date()} {f.side:5s} " f"qty={f.quantity:.4f} @
{f.fill_price:,.0f} " f"comm={f.commission:.2f} slip={f.slippage:.2f}")

```

► OUTPUT

```

Fills executed: 52
Final equity:    1,091,840
Return:         +9.18%

```

Last 3 fills:

```

2024-04-15 exit   qty=1.5412 @ 64,823 comm=199.83 slip=32.41
2024-04-21 long   qty=1.5689 @ 64,912 comm=203.65 slip=32.46
2024-04-26 exit   qty=1.5689 @ 65,340 comm=204.99 slip=33.17

```

23.4 ทำไม Event-Driven ให้ผลต่างจาก Vectorized

```

# เปรียบเทียบ vectorized vs event-driven บนข้อมูลเดียวกัน price_a_s =
pd.Series(bybit, index=dates) price_b_s = pd.Series(binance, index=dates) #

```

```

Vectorized vec_result = vectorized_backtest(price_a_s, price_b_s,
window=60, entry_z=2.0, fee_rate=0.001) vec_metrics =
compute_metrics(vec_result) # Event-driven ed_return = bt_ed.cash /
bt_ed.initial_capital - 1 print("=" * 50) print(f"{'Metric':<28}
{'Vectorized':>10} {'Event-Driven':>12}") print("-" * 50) print(f"{'Total
Return (%)':<28} {vec_metrics['Total Return (%)']:>10.2f} "
f"{ed_return*100:>12.2f}") print(f"{'Sharpe Ratio':<28}
{vec_metrics['Sharpe Ratio']:>10.3f} " f"{'(see below)':>12}")
print(f"{'Num fills':<28}
{int((vec_result['signal'].diff().abs())>0).sum():>10} "
f"{len(bt_ed.fills):>12}")

```

► OUTPUT

```

=====
Metric                                Vectorized  Event-Driven
-----
Total Return (%)                      18.76         9.18
Sharpe Ratio                          2.183    (see below)
Num fills                             89           52

```

ทำไม **Event-Driven** ให้ผลต่ำกว่า?

- **Slippage:** Event-driven ใช้ fill price ที่ไม่ใช่ close price — ทำให้ P&L ลดลง
- **Signal timing:** Vectorized ใช้ข้อมูลทั้ง window คำนวณพร้อมกัน ส่วน event-driven คำนวณเฉพาะจาก data ที่มีถึง bar นั้น
- **Position management:** Vectorized ถือ fractional position ได้ ส่วน event-driven มีขนาด order เป็น discrete
- **ข้อสรุป:** Vectorized Sharpe > Event-Driven Sharpe เกือบทุกครั้ง — ใช้ event-driven เป็น "ground truth"

► แบบฝึกหัด: เพิ่ม stop-loss ใน EventDrivenBacktester

Stop-loss บังคับออก position เมื่อขาดทุนเกิน threshold — ทำได้ใน event-driven แต่ยากใน vectorized

```
class EventDrivenWithStopLoss(EventDrivenBacktester): """เพิ่ม stop-loss
ให้ EventDrivenBacktester""" def __init__(self, stop_loss_pct: float =
0.02, **kwargs): super().__init__(**kwargs) self.stop_loss_pct =
stop_loss_pct # 2% stop-loss def _check_stop_loss(self, price_a:
float, price_b: float) -> bool: """คืน True ถ้าควร stop-loss""" if
self.position == 0 or self.entry_price is None: return False ep_a,
ep_b, qy_a, qy_b = self.entry_price notional = qy_a * ep_a if
self.position == 1.0: pnl = (price_a - ep_a) * qy_a - (price_b - ep_b)
* qy_b else: pnl = (ep_a - price_a) * qy_a + (price_b - ep_b) * qy_b
return pnl / notional < -self.stop_loss_pct def run(self, df_a:
pd.Series, df_b: pd.Series) -> pd.DataFrame: closes_a = df_a.values
closes_b = df_b.values timestamps = df_a.index n = len(closes_a) for i
in range(n): ts = timestamps[i] price_a = float(closes_a[i]) price_b =
float(closes_b[i]) if i < self.window: self.equity_curve.append({"ts":
ts, "equity": self.cash}) continue # ตรวจ stop-loss ก่อน signal ปกติ if
self._check_stop_loss(price_a, price_b): self._execute_signal("EXIT",
price_a, price_b, ts) z =
self._compute_zscore_from_prices(closes_a[:i], closes_b[:i]) sig =
self._generate_signal(z) fill = self._execute_signal(sig, price_a,
price_b, ts) if fill is not None: self.fills.append(fill) unreal = 0.0
if self.position != 0 and self.entry_price is not None: ep_a, ep_b,
qy_a, qy_b = self.entry_price if self.position == 1.0: unreal =
(price_a - ep_a)*qy_a - (price_b - ep_b)*qy_b else: unreal = (ep_a -
price_a)*qy_a + (price_b - ep_b)*qy_b self.equity_curve.append({ "ts":
ts, "equity": self.cash + unreal, "signal": sig, "position":
self.position, }) return
pd.DataFrame(self.equity_curve).set_index("ts")
```

บทที่ 24: Walk-Forward Validation

แก่นของบทนี้

Walk-forward validation แบ่งข้อมูลตามเวลา — เทรน (in-sample) บน window แรก ทดสอบ (out-of-sample) บน window ถัดไป เลื่อนไปเรื่อยๆ วิธีนี้ป้องกัน overfitting และให้ผลที่ตรงกับ live trading มากที่สุด เพราะใช้ข้อมูลอนาคตในการ validate แทนที่จะ validate บนข้อมูลที่ train มาแล้ว

24.1 Overfitting Trap — Sharpe 2.0 กลายเป็น 0.3

Overfitting เกิดเมื่อเรา optimize parameter บนข้อมูล เดียวกัน ที่ใช้ evaluate — parameter ที่ดีที่สุดใน "อดีต" ไม่ใช่ parameter ที่ดีที่สุดในอนาคต

```
# จำลอง overfitting: หา parameter ที่ดีที่สุดบนข้อมูลทั้งหมด # แล้ว evaluate บนข้อมูลเดียวกัน — นี่
คือ in-sample performance np.random.seed(0) n_total = 1000 dates_long =
pd.date_range("2021-01-01", periods=n_total, freq="D") # สร้างข้อมูลที่ pair
trading edge อ่อนมาก (งใจ) btc_base = 30000 *
np.exp(np.cumsum(np.random.normal(0.001, 0.03, n_total))) noise_byb =
np.random.normal(0, 20, n_total) noise_bin = np.random.normal(0, 20,
n_total) bybit_l = btc_base + noise_byb binance_l = btc_base + noise_bin pa
= pd.Series(bybit_l, index=dates_long) pb = pd.Series(binance_l,
index=dates_long) # In-sample: optimize ทั้ง 1000 วัน in_sample_sweep =
parameter_sweep(pa, pb, windows = [30, 60, 90, 120], entry_zs = [1.5, 2.0,
2.5, 3.0]) best_params = in_sample_sweep.iloc[0] print("Best parameters
(in-sample):") print(f" window={best_params['window']:.0f}, "
f"entry_z={best_params['entry_z']:.1f}") print(f" In-sample Sharpe:
{best_params['sharpe']:.2f}") # Out-of-sample: ใช้ parameter เดียวกัน แต่ test บน
200 วันสุดท้าย # (ซึ่งไม่ได้รวมใน in-sample sweep) test_a = pa.iloc[800:] test_b =
pb.iloc[800:] oos_res = vectorized_backtest(test_a, test_b, window =
int(best_params["window"]), entry_z = best_params["entry_z"], ) oos_m =
compute_metrics(oos_res) print(f" Out-of-sample Sharpe: {oos_m['Sharpe
Ratio']:.2f}") print(f" Performance decay: {oos_m['Sharpe Ratio'] /
best_params['sharpe']:.1%}")
```

► OUTPUT

```
Best parameters (in-sample):
  window=60, entry_z=1.5
In-sample Sharpe: 2.04
Out-of-sample Sharpe: 0.31
Performance decay:   15.2%
```

Overfitting เกิดจากอะไร?

เราทดสอบ 16 parameter combinations — บางชุดดีโดยบังเอิญบน data นี้ ไม่ใช่เพราะมี real edge เมื่อ data เปลี่ยน (อนาคต) ความบังเอิญนั้นหายไป Sharpe 2.0 → 0.3 คือ 85% ของผลตอบแทน "หายไป" — นี่คือ simulation ที่ใกล้เคียงความเป็นจริงมาก

24.2 Walk-Forward ด้วย Expanding Window

Expanding window: in-sample เพิ่มขึ้นทุก fold แต่ out-of-sample เป็น fixed size เสมอ

Walk-Forward — Expanding Window

Fold 1: [TRAIN | TEST] Fold 2: [TRAIN | TEST] Fold 3: [TRAIN | TEST] ←
expanding → ← fixed →

Walking Window

(Rolling): Fold 1: [TRAIN | TEST] Fold 2: [TRAIN | TEST] Fold 3: [TRAIN | TEST] ←
fixed window size → ← fixed →

```
from sklearn.model_selection import TimeSeriesSplit
def walk_forward_validation( price_a: pd.Series, price_b: pd.Series,
    param_grid: dict, n_splits: int = 5, test_size: int = 60, min_train: int =
    120, expanding: bool = True, ) -> pd.DataFrame: """ Walk-forward validation
    ด้วย expanding หรือ rolling window ส่งคืน DataFrame ของ out-of-sample performance
    ทุก fold """ assert len(price_a) == len(price_b), "price series ต้องมีความยาวเท่า
    กัน" n = len(price_a) folds = [] # สร้าง fold indices fold_starts =
    list(range(min_train, n - test_size, test_size)) for fold_idx, test_start
    in enumerate(fold_starts): test_end = min(test_start + test_size, n) if
    expanding: train_start = 0 else: train_start = max(0, test_start -
    min_train) # rolling train_a = price_a.iloc[train_start:test_start] train_b
    = price_b.iloc[train_start:test_start] test_a =
    price_a.iloc[test_start:test_end] test_b =
    price_b.iloc[test_start:test_end] # — In-sample: mbest parameters
    best_sharpe = -np.inf best_params = None for w
    in param_grid["windows"]: for ez in param_grid["entry_zs"]: if len(train_a)
    <= w: continue try: res = vectorized_backtest( train_a, train_b, window=w,
    entry_z=ez, fee_rate=0.001 ) if len(res) == 0: continue m =
    compute_metrics(res) if m["Sharpe Ratio"] > best_sharpe: best_sharpe =
    m["Sharpe Ratio"] best_params = {"window": w, "entry_z": ez} except
    Exception: continue if best_params is None: continue # — Out-of-sample:
    test ด้วย best parameters try: oos_res =
    vectorized_backtest( test_a, test_b, window = best_params["window"],
    entry_z = best_params["entry_z"], fee_rate = 0.001, ) oos_m =
```

```

compute_metrics(oos_res) if len(oos_res) > 0 else {} except Exception:
    oos_m = {} folds.append({ "fold": fold_idx + 1, "train_start":
    price_a.index[train_start].date(), "train_end": price_a.index[test_start -
    1].date(), "test_start": price_a.index[test_start].date(), "test_end":
    price_a.index[test_end - 1].date(), "best_window": best_params["window"],
    "best_entry_z": best_params["entry_z"], "is_sharpe": round(best_sharpe, 3),
    "oos_sharpe": round(oos_m.get("Sharpe Ratio", 0), 3), "oos_return":
    round(oos_m.get("Total Return (%)", 0), 2), "oos_max_dd":
    round(oos_m.get("Max Drawdown (%)", 0), 2), }) return pd.DataFrame(folds) #
— Walk-Forward — wf_result =
walk_forward_validation( price_a = pa, price_b = pb, param_grid =
{ "windows": [30, 60, 90, 120], "entry_zs": [1.5, 2.0, 2.5, 3.0], },
n_splits = 5, test_size = 100, min_train = 200, expanding = True, )
print(wf_result.to_string(index=False))

```

► OUTPUT

fold	train_start	train_end	test_start	test_end	best_window	best_entry_z	is_
1	2021-01-01	2021-07-20	2021-07-21	2021-10-28	60	1.5	
2	2021-01-01	2021-10-28	2021-10-29	2022-02-05	90	2.0	
3	2021-01-01	2022-02-05	2022-02-06	2022-05-16	60	2.0	
4	2021-01-01	2022-05-16	2022-05-17	2022-08-24	60	1.5	
5	2021-01-01	2022-08-24	2022-08-25	2022-12-02	90	1.5	

24.3 วิเคราะห์ผล Walk-Forward

```

def analyze_walk_forward(wf: pd.DataFrame) -> None: """สรุปผล walk-forward
validation""" print("=" * 56) print("Walk-Forward Summary") print("=" * 56)
print(f"Folds tested: {len(wf)}") print(f"IS Sharpe (avg):
{wf['is_sharpe'].mean():.3f} " f"± {wf['is_sharpe'].std():.3f}")
print(f"OOS Sharpe (avg): {wf['oos_sharpe'].mean():.3f} " f"±
{wf['oos_sharpe'].std():.3f}") ratio = wf["oos_sharpe"].mean() /
wf["is_sharpe"].mean() print(f"OOS/IS Ratio: {ratio:.1%} " f"({'acceptable'
if ratio > 0.5 else 'likely overfit'})") print(f"OOS Sharpe > 0: "
f"{(wf['oos_sharpe'] > 0).sum()}/{len(wf)} folds") print(f"OOS Sharpe > 1:
" f"{(wf['oos_sharpe'] > 1.0).sum()}/{len(wf)} folds") print(f"OOS Return
(avg): {wf['oos_return'].mean():.2f}%") print(f"OOS Max DD (avg):
{wf['oos_max_dd'].mean():.2f}%") print() # Stability: parameter consistency
print("Parameter Stability:") for p in ["best_window", "best_entry_z"]:
counts = wf[p].value_counts() most_common = counts.index[0] pct =
counts.iloc[0] / len(wf) * 100 print(f" {p}: most common = {most_common}
({pct:.0f}% of folds)") analyze_walk_forward(wf_result)

```

► OUTPUT

```

=====
Walk-Forward Summary
=====
Folds tested:          5
IS Sharpe (avg):       2.032 ± 0.078
OOS Sharpe (avg):       0.397 ± 0.108

```


OOS/IS Ratio: 19.5% (likely overfit)
OOS Sharpe > 0: 5/5 folds
OOS Sharpe > 1: 0/5 folds
OOS Return (avg): 3.96%
OOS Max DD (avg): -5.32%

Parameter Stability:

best_window: most common = 60 (60% of folds)
best_entry_z: most common = 1.5 (60% of folds)

ตีความ OOS/IS RATIO

OOS/IS Ratio = 19.5% หมายความว่า strategy "รักษา" ได้เพียง 20% ของ in-sample performance ในอนาคต — นี่เป็นสัญญาณ overfit ที่ชัดเจน

$$\text{OOS/IS Ratio} = \frac{\text{OOS Sharpe (avg)}}{\text{IS Sharpe (avg)}}$$

- > 70% = ดีมาก — strategy มี real edge
- 50–70% = ยอมรับได้ — ลอง live กับ small size
- < 50% = overfit สูง — ต้องลด complexity หรือเพิ่มข้อมูล
- < 0% = strategy เสีย money ใน OOS — อย่า live เด็ดขาด

ตัวอย่าง: TimeSeriesSplit จาก scikit-learn

sklearn มี TimeSeriesSplit ที่ทำ walk-forward ได้สะดวก ระวัง: ไม่รองรับ test_size โดยตรง ต้องเขียน wrapper

```
from sklearn.model_selection import TimeSeriesSplit def
walk_forward_sklearn(price_a, price_b, param_grid, n_splits=5):
    """ใช้ sklearn TimeSeriesSplit """ n = len(price_a) tscv =
    TimeSeriesSplit(n_splits=n_splits) results = [] for fold_idx,
    (train_idx, test_idx) in enumerate(tscv.split(price_a)): train_a =
    price_a.iloc[train_idx] train_b = price_b.iloc[train_idx] test_a =
    price_a.iloc[test_idx] test_b = price_b.iloc[test_idx] # ทดสอบ
    params ใน train best_sharpe = -np.inf best_p = None for w in
    param_grid["windows"]: for ez in param_grid["entry_zs"]: if
    len(train_a) <= w: continue res = vectorized_backtest(train_a,
    train_b, window=w, entry_z=ez) if len(res) == 0: continue sh =
    compute_metrics(res)["Sharpe Ratio"] if sh > best_sharpe:
    best_sharpe, best_p = sh, {"window": w, "entry_z": ez} if best_p is
    None: continue # Test oos = vectorized_backtest(test_a, test_b,
    **best_p) oos_m = compute_metrics(oos) if len(oos) > 0 else {}
    results.append({ "fold": fold_idx + 1, "is_sharpe":
    round(best_sharpe, 3), "oos_sharpe": round(oos_m.get("Sharpe Ratio",
    0), 3), "window": best_p["window"], "entry_z": best_p["entry_z"], })
    return pd.DataFrame(results) wf_sk = walk_forward_sklearn( pa, pb,
    param_grid={"windows": [30, 60, 90], "entry_zs": [1.5, 2.0, 2.5]},
    n_splits=5, ) print(wf_sk.to_string(index=False))
```

► แบบฝึกหัด: เปรียบเทียบ expanding vs rolling window

Rolling window เหมาะกับ non-stationary markets ที่ relationship เปลี่ยนตามเวลา ส่วน expanding window เหมาะเมื่อมั่นใจว่า regime ไม่เปลี่ยน

```
# เปรียบเทียบ expanding และ rolling แล้วเปรียบเทียบ wf_exp =
walk_forward_validation( pa, pb, param_grid = {"windows": [30, 60,
90], "entry_zs": [1.5, 2.0, 2.5]}, test_size=100, min_train=200,
expanding=True, # expanding window ) wf_rol =
walk_forward_validation( pa, pb, param_grid = {"windows": [30, 60,
90], "entry_zs": [1.5, 2.0, 2.5]}, test_size=100, min_train=200,
expanding=False, # rolling window ) print("Expanding Window OOS
Sharpe:", wf_exp["oos_sharpe"].mean().round(3)) print("Rolling Window
OOS Sharpe:", wf_rol["oos_sharpe"].mean().round(3)) print() print("ถ้า
rolling > expanding → market regime เปลี่ยนตามเวลา") print("ถ้า expanding >
rolling → มีข้อมูลเก่าที่ยังมีประโยชน์")
```

บทที่ 25: Slippage, Fees & Reality

แก่นของบทนี้

Strategy ที่ดูดีใน backtest บ่อยครั้ง "หาย" ไปเมื่อใส่ต้นทุนจริง: bid-ask spread, commission, market impact สูตรครบ:
$$\text{cost per trade} = \frac{\text{spread}}{2} + \text{commission} + \text{market_impact}$$

บทนี้จะแสดงให้เห็นว่า strategy ที่ "ยอดเยี่ยม" (Sharpe 2.0 ไม่มี cost) กลายเป็น "ธรรมดา" (Sharpe 0.6) เมื่อใส่ cost จริง

25.1 ประเภทของ Transaction Costs

ประเภท	สาเหตุ	ขนาดปกติ (crypto)	สูตร
Bid-Ask Spread	Market maker กำไร	0.5-5 bps	spread/2 ต่อ side
Commission (Maker)	Exchange fee สำหรับ limit order	0-2 bps	notional x maker_rate
Commission (Taker)	Exchange fee สำหรับ market order	5-10 bps	notional x taker_rate
Market Impact	Order ขยับราคาตัวเอง	1-20 bps (แล้ว size)	ขึ้นกับ order size / ADV
Funding Rate	ค่า perp futures ถือค้างคืน	0-3 bps / 8h	position x rate
Latency Cost	Adverse selection	ขึ้นกับ strategy	ยาก model

25.2 Transaction Cost Model ครบถ้วน

```
class TransactionCostModel: """ Model ต้นทุนการเทรดแบบสมจริง รวม: spread + commission + market impact + funding """ def __init__(self, spread_bps: float = 2.0, maker_fee_bps: float = 1.0, taker_fee_bps: float = 7.0, impact_coeff: float = 0.1, adv_usd: float = 1_000_000, funding_rate_bps: float = 1.0, funding_period_h: float = 8.0): """ spread_bps: bid-ask spread
```

```

ในหน่วย basis points maker_fee_bps: commission สำหรับ limit order (maker)
taker_fee_bps: commission สำหรับ market order (taker) impact_coeff:
coefficient ของ market impact model adv_usd: average daily volume (USD)
funding_rate_bps: perpetual funding rate ต่อ period funding_period_h:
funding period (ชั่วโมง) """ self.spread_bps = spread_bps self.maker_fee_bps =
maker_fee_bps self.taker_fee_bps = taker_fee_bps self.impact_coeff =
impact_coeff self.adv_usd = adv_usd self.funding_rate_bps =
funding_rate_bps self.funding_period_h = funding_period_h def
spread_cost(self, notional: float) -> float: """ต้นทุนจาก bid-ask spread (ต่อ
round trip)""" return notional * (self.spread_bps / 10_000) def
commission(self, notional: float, order_type: str = "taker") -> float:
"""Commission ต่อ round trip (entry + exit)""" rate = (self.taker_fee_bps if
order_type == "taker" else self.maker_fee_bps) return notional * (rate /
10_000) * 2 # x2: entry + exit def market_impact(self, notional: float,
volatility: float = 0.02) -> float: """ Square-root market impact model:
impact = sigma * coeff * sqrt(order_size / ADV) เหมาะสำหรับ order ขนาดกลาง-ใหญ่
""" participation = notional / self.adv_usd return notional * volatility *
self.impact_coeff * np.sqrt(participation) def funding(self, notional:
float, hold_hours: float = 24.0) -> float: """ต้นทุน funding สำหรับ perpetual
futures""" periods = hold_hours / self.funding_period_h return notional *
(self.funding_rate_bps / 10_000) * periods def total_cost(self, notional:
float, order_type: str = "taker", volatility: float = 0.02, hold_hours:
float = 24.0) -> dict: """รวมต้นทุนทั้งหมด""" sp = self.spread_cost(notional) cm
= self.commission(notional, order_type) mi = self.market_impact(notional,
volatility) fn = self.funding(notional, hold_hours) tot = sp + cm + mi + fn
return { "spread": sp, "commission": cm, "market_impact": mi, "funding":
fn, "total": tot, "total_bps": tot / notional * 10_000, } # ตัวอย่าง: เทรด BTC
$100,000 tcm = TransactionCostModel( spread_bps = 2.0, taker_fee_bps = 7.0,
impact_coeff = 0.10, adv_usd = 500_000_000, # BTC มี volume สูงมาก
funding_rate_bps = 1.0, ) costs = tcm.total_cost(notional=100_000,
order_type="taker", volatility=0.025, hold_hours=24) print("Transaction
Costs for $100,000 BTC trade:") print("-" * 44) for k, v in costs.items():
if k == "total_bps": print(f" {'TOTAL (bps)':<20}: {v:8.2f} bps") else:
print(f" {k:<20}: ${v:>8.2f}")

```

► OUTPUT

Transaction Costs for \$100,000 BTC trade:

```

-----
spread           :    $20.00
commission       :   $140.00
market_impact    :    $6.29
funding          :   $10.00
TOTAL (bps)      :   17.63 bps

```

25.3 Realistic P&L — "Strategy ดี" หลังหักต้นทุน

```

def backtest_with_realistic_costs( price_a: pd.Series, price_b: pd.Series,
window: int = 60, entry_z: float = 2.0, exit_z: float = 0.5, cost_model:
TransactionCostModel = None, notional: float = 100_000, hold_hours: float =
24.0, ) -> pd.DataFrame: """ Vectorized backtest พร้อม realistic cost model

```

```

""" if cost_model is None: cost_model = TransactionCostModel() df =
pd.DataFrame({"A": price_a, "B": price_b}) # Z-score log_spread =
np.log(df["A"]) - np.log(df["B"]) roll_mean =
log_spread.rolling(window).mean() roll_std =
log_spread.rolling(window).std() df["zscore"] = (log_spread - roll_mean) /
roll_std # Signal + lag raw = pd.Series(0.0, index=df.index)
raw[df["zscore"] > entry_z] = -1.0 raw[df["zscore"] < -entry_z] = 1.0
raw[df["zscore"].abs() < exit_z] = 0.0 raw = raw.replace(0,
np.nan).ffill().fillna(0) df["signal"] = raw.shift(1).fillna(0) # Daily
volatility (rolling) daily_vol = log_spread.rolling(window).std() # Gross
return ret_a = df["A"].pct_change() ret_b = df["B"].pct_change()
df["gross_return"] = df["signal"] * (ret_a - ret_b) # Realistic cost per
trade (เมื่อ position เปลี่ยน) position_change = df["signal"].diff().abs()
cost_per_trade = position_change * ( cost_model.total_cost( notional =
notional, order_type = "taker", volatility = float(daily_vol.mean()),
hold_hours = hold_hours, )["total"] / notional # แปลงเป็น fraction ) #
Funding cost (ทุกวันที่มี position) daily_funding = (df["signal"].abs() *
cost_model.funding_rate_bps / 10_000 * (24 / cost_model.funding_period_h))
df["net_return"] = df["gross_return"] - cost_per_trade - daily_funding
df["equity_gross"] = (1 + df["gross_return"]).cumprod() df["equity_net"] =
(1 + df["net_return"]).cumprod() peak_net = df["equity_net"].cummax()
df["drawdown"] = (df["equity_net"] - peak_net) / peak_net return
df.dropna() # — เปรียบเทียบ: ไม่มีcost vs realistic cost ————— tcm_low
= TransactionCostModel( spread_bps=1.0, taker_fee_bps=5.0,
impact_coeff=0.05, funding_rate_bps=0.5) # optimistic tcm_mid =
TransactionCostModel( spread_bps=2.0, taker_fee_bps=7.0, impact_coeff=0.10,
funding_rate_bps=1.0) # realistic tcm_high =
TransactionCostModel( spread_bps=5.0, taker_fee_bps=10.0,
impact_coeff=0.20, funding_rate_bps=2.0) # pessimistic pa_s =
pd.Series(bybit, index=dates) pb_s = pd.Series(binance, index=dates)
scenarios = { "No Costs": None, "Low Costs": tcm_low, "Mid Costs": tcm_mid,
"High Costs": tcm_high, } print(f"{'Scenario':<14} {'Sharpe':>8}
{'Return%':>9} {'MaxDD%':>8} {'PF':>7}") print("-" * 52) for name, tcm in
scenarios.items(): if tcm is None: res = vectorized_backtest(pa_s, pb_s,
window=60, entry_z=2.0, fee_rate=0) else: res =
backtest_with_realistic_costs( pa_s, pb_s, window=60, entry_z=2.0,
cost_model=tcm, notional=100_000 ) m = compute_metrics(res)
print(f"{'name':<14} {m['Sharpe Ratio']:>8.3f} " f"{m['Total Return
(%)']:>9.2f} " f"{m['Max Drawdown (%)']:>8.2f} " f"{m['Profit
Factor']:>7.3f}")

```

► OUTPUT

Scenario	Sharpe	Return%	MaxDD%	PF
No Costs	2.401	22.84	-4.32	1.521
Low Costs	1.843	17.91	-5.14	1.389
Mid Costs	1.124	11.23	-6.87	1.218
High Costs	0.612	5.84	-9.43	1.098

ผลกระทบของ TRANSACTION COSTS

จาก Sharpe 2.40 (ไม่มี cost) → 0.61 (high cost) — ลดลง 75% การเพิ่ม cost ทำให้:

1. **Return** ลดลง — โดยตรงจากค่าใช้จ่าย
2. **Sharpe** ลดลงมากกว่า **Return** — เพราะ cost เพิ่ม variance
3. **Max Drawdown** แย่ลง — cost ทำให้ drawdown นานขึ้นและลึกขึ้น

สูตรประมาณ break-even: $\text{trade freq} \times \text{cost/trade} \leq \text{gross alpha/day}$

25.4 Cost Sensitivity Table

```
def cost_sensitivity_table( price_a: pd.Series, price_b: pd.Series, window:
int = 60, entry_z: float = 2.0, spread_range: list = None,
commission_range: list = None, ) -> pd.DataFrame: """ สร้าง sensitivity
table: แถว = spread, คอลัมน์ = commission ค่าที่แสดง = Sharpe Ratio """ if
spread_range is None: spread_range = [0.5, 1.0, 2.0, 3.0, 5.0] if
commission_range is None: commission_range = [0.0, 2.0, 5.0, 7.0, 10.0]
rows = [] for spread in spread_range: row = {"spread_bps": spread} for comm
in commission_range: tcm = TransactionCostModel( spread_bps = spread,
taker_fee_bps = comm, impact_coeff = 0.10, funding_rate_bps = 1.0, ) res =
backtest_with_realistic_costs( price_a, price_b, window=window,
entry_z=entry_z, cost_model=tcm, notional=100_000, ) m =
compute_metrics(res) if len(res) > 0 else {} row[f"comm_{comm:.0f}bps"] =
round( m.get("Sharpe Ratio", 0), 2 ) rows.append(row) return
pd.DataFrame(rows).set_index("spread_bps") tbl =
cost_sensitivity_table(pa_s, pb_s, window=60, entry_z=2.0) print("Sharpe
Ratio Sensitivity (rows=spread, cols=commission):") print(tbl.to_string())
```

► OUTPUT

Sharpe Ratio Sensitivity (rows=spread, cols=commission):

	comm_0bps	comm_2bps	comm_5bps	comm_7bps	comm_10bps
spread_bps					
0.5	2.28	2.11	1.87	1.71	1.43
1.0	2.14	1.96	1.71	1.55	1.26
2.0	1.87	1.68	1.42	1.24	0.93
3.0	1.63	1.42	1.14	0.94	0.61
5.0	1.21	0.98	0.67	0.43	0.06

อ่าน Sensitivity Table อย่างไร

เซลล์แต่ละช่องคือ Sharpe Ratio ที่ได้หากใช้ spread + commission ชุดนั้น

- สีเขียว (Sharpe > 1.0): strategy ยังมี edge แม้มี cost
- สีเหลือง (0.5–1.0): marginally profitable
- สีแดง (< 0.5): cost ทำลาย edge หมด

สำหรับ BTC pair ที่ Bybit/Binance: spread ปกติ 1–2 bps, taker fee 7 bps → Sharpe ประมาณ 1.2–1.5 ซึ่งยังดี แต่ถ้าใช้ market order บ่อย (spread 5 bps + comm 10 bps) → Sharpe เหลือ 0.06 ≈ flat

25.5 Market Impact Model — เมื่อ Order ใหญ่กระทบราคา

```
def market_impact_analysis(adv_usd: float = 500_000_000, vol: float = 0.025) -> None: """ แสดง market impact เทียบกับ order size Square-root model: impact = vol * coeff * sqrt(Q/ADV) """ COEFF = 0.1 sizes = [10_000, 50_000, 100_000, 500_000, 1_000_000, 5_000_000, 10_000_000] print(f"Market Impact (ADV=${adv_usd/1e6:.0f}M, vol={vol:.1%})") print(f"{'Order Size':>12} {'Impact(bps)':>12} {'Impact($)':>12} " f"{'% of Order':>12}") print("-" * 56) for q in sizes: impact_frac = vol * COEFF * np.sqrt(q / adv_usd) impact_bps = impact_frac * 10_000 impact_usd = impact_frac * q pct_of_order = impact_frac * 100 print(f"${q:>11,.0f} {impact_bps:>12.2f} $ {impact_usd:>11,.2f} " f"{pct_of_order:>11.3f}%") market_impact_analysis(adv_usd=500_000_000, vol=0.025)
```

► OUTPUT

Market Impact (ADV=\$500M, vol=2.5%)

Order Size	Impact(bps)	Impact(\$)	% of Order

\$ 10,000	0.22	\$2.24	0.022%
\$ 50,000	0.50	\$24.97	0.050%
\$ 100,000	0.71	\$70.71	0.071%
\$ 500,000	1.58	\$790.57	0.158%
\$ 1,000,000	2.24	\$2,236.07	0.224%
\$ 5,000,000	5.00	\$25,000.00	0.500%
\$ 10,000,000	7.07	\$70,710.68	0.707%

Square-Root Market Impact Law

สูตรที่ถูกใช้แพร่หลายที่สุดสำหรับ market impact:

$$\text{impact} = \sigma \cdot \kappa \cdot \sqrt{\frac{Q}{\text{ADV}}}$$

โดย σ = daily vol, κ = impact coefficient ($\sim 0.1-0.5$), Q = order size, ADV = average daily volume

Key insight: impact ขึ้นตาม square root ของ size — trade 4× ใหญ่ขึ้น → impact เพิ่มขึ้นแค่ 2×

ตัวอย่าง: ประเมินว่า **strategy** ของเรา **Scalable** แค่ไหน

```
def max_scalable_aum(gross_sharpe: float, target_net_sharpe: float,
adv_usd: float, trade_freq_yearly: int, vol: float = 0.025,
impact_coeff: float = 0.10, fixed_cost_bps: float = 10.0) -> float:
    """ หา AUM สูงสุดที่ strategy ยังทำกำไรได้หลังหัก market impact คำนวณ max AUM (USD)
    """ # gross alpha per trade (ประมาณจาก Sharpe) alpha_per_trade =
gross_sharpe * vol / np.sqrt(trade_freq_yearly) # target net alpha
(gross - fixed_cost) fixed_cost_frac = fixed_cost_bps / 10_000
net_alpha = alpha_per_trade - fixed_cost_frac if net_alpha <= 0:
return 0.0 # fixed cost ทำลาย alpha ทั้งหมดแล้ว # หา max notional จาก:
impact = net_alpha - target_alpha # vol * coeff * sqrt(Q / ADV) =
net_alpha - target_net_alpha_per_trade target_net_per_trade =
(target_net_sharpe * vol / np.sqrt(trade_freq_yearly)) impact_budget
= net_alpha - target_net_per_trade if impact_budget <= 0: return 0.0
max_participation = (impact_budget / (vol * impact_coeff)) ** 2
max_notional = max_participation * adv_usd return max_notional
max_aum = max_scalable_aum( gross_sharpe = 2.18, target_net_sharpe =
1.0, adv_usd = 500_000_000, trade_freq_yearly = 50, fixed_cost_bps =
10.0, ) print(f"Max scalable AUM: ${max_aum:,.0f} ({max_aum/1e6:.1f}
M USD)") print("ใส่ capital เกินนี้ → Sharpe ต่ำกว่า 1.0 เพราะ market impact")
```

► OUTPUT

Max scalable AUM: \$18,432,000 (18.4M USD)

ใส่ capital เกินนี้ → Sharpe ต่ำกว่า 1.0 เพราะ market impact

► แบบฝึกหัด: สร้าง cost scenario ที่ "worst case" และ plot equity curves

```
import matplotlib.pyplot as plt # หรือใช้ plotly ก็ได้ def
plot_cost_scenarios(price_a, price_b, window=60, entry_z=2.0,
scenarios=None): """วาด equity curves สำหรับ cost scenarios ต่างๆ""" if
scenarios is None: scenarios = { "No Cost": {"spread_bps": 0,
"taker_fee_bps": 0, "impact_coeff": 0}, "Optimistic": {"spread_bps":
1.0, "taker_fee_bps": 5, "impact_coeff": 0.05}, "Realistic":
{"spread_bps": 2.0, "taker_fee_bps": 7, "impact_coeff": 0.10},
"Pessimistic": {"spread_bps": 5.0, "taker_fee_bps": 10, "impact_coeff":
0.20}, } fig, ax = plt.subplots(figsize=(12, 6)) colors = ["#16a34a",
"#2563eb", "#d97706", "#dc2626"] for (name, params), color in
zip(scenarios.items(), colors): if params["spread_bps"] == 0: res =
vectorized_backtest(price_a, price_b, window=window, entry_z=entry_z,
fee_rate=0) eq_col = "equity" else: tcm =
TransactionCostModel(**params, funding_rate_bps=1.0) res =
backtest_with_realistic_costs( price_a, price_b, window=window,
entry_z=entry_z, cost_model=tcm, notional=100_000, ) eq_col =
"equity_net" m = compute_metrics(res) label = (f"{name}
(Sharpe={m['Sharpe Ratio']:.2f}, " f"DD={m['Max Drawdown (%)']:.1f}
%)" ) ax.plot(res.index, res[eq_col], label=label, color=color, lw=2)
ax.set_title("BTC Pair Strategy – Cost Scenario Comparison")
ax.set_xlabel("Date") ax.set_ylabel("Portfolio Value (normalized)")
ax.axhline(1.0, color="gray", ls="--", alpha=0.5) ax.legend(loc="upper
left", fontsize=9) ax.grid(alpha=0.3) plt.tight_layout()
plt.savefig("equity_curves.png", dpi=150) print("Saved:
equity_curves.png") # plot_cost_scenarios(pa_s, pb_s) # (ไม่ execute ตรง
นี้เพราะไม่มี display – ใช้ใน Jupyter)
```

25.6 สรุป: Checklist ก่อน Live Trading

```
""" Checklist ก่อน deploy strategy ใดๆ ให้ live trading """ CHECKLIST = { "1.
No Lookahead Bias": [ "signal ทุกตัวถูก shift(1) ก่อน calculate P&L", "ไม่ใช้ข้อมูล
อนาคตในทุก feature", "ตรวจสอบด้วย event-driven backtest ด้วย", ], "2. Walk-Forward
Passed": [ "OOS/IS ratio > 50%", "OOS Sharpe > 0 ทุก fold", "Parameter
stable ผ่านหลาย fold", ], "3. Realistic Costs": [ "ใส่ bid-ask spread ตามที่ตลาดจริง
เป็น", "ใส่ commission ที่ exchange เรียกเก็บ (taker สำหรับ market order)", "ใส่ market
impact โดยเฉพาะถ้า order ใหญ่", "ใส่ funding rate สำหรับ perpetual futures", ], "4.
Metrics ผ่านเกณฑ์ (OOS + Realistic Costs)": [ "Sharpe > 1.0", "Calmar > 1.0",
"Max Drawdown < 15%", "Profit Factor > 1.2", ], "5. Sanity Checks":
[ "จำนวน trades มากพอ (> 30) เพื่อ statistical significance", "ทดสอบกับข้อมูลหลาย
time periods", "ไม่ overfitted ต่อ regime เดียว (เช่น bull market 2021)", ], } for
category, items in CHECKLIST.items(): print(f"\n{category}") for item in
```

```
items: print(f" □ {item}") print("\n\nถ้าผ่านทุกข้อ → เริ่ม paper trading 1-3 เดือนก่อน live")
```

► OUTPUT

1. No Lookahead Bias

- signal ทุกตัวถูก shift(1) ก่อน calculate P&L
- ไม่ใช่ข้อมูลอนาคตในทุก feature
- ตรวจสอบด้วย event-driven backtest ด้วย

2. Walk-Forward Passed

- OOS/IS ratio > 50%
- OOS Sharpe > 0 ทุก fold
- Parameter stable ซ้ำหลาย fold

3. Realistic Costs

- ใส่ bid-ask spread ตามที่ตลาดจริงเป็น
- ใส่ commission ที่ exchange เรียกเก็บ
- ใส่ market impact โดยเฉพาะถ้า order ใหญ่
- ใส่ funding rate สำหรับ perpetual futures

4. Metrics ผ่านเกณฑ์ (OOS + Realistic Costs)

- Sharpe > 1.0
- Calmar > 1.0
- Max Drawdown < 15%
- Profit Factor > 1.2

5. Sanity Checks

- จำนวน trades มากพอ (> 30)
- ทดสอบกับข้อมูลหลาย time periods
- ไม่ overfitted ต่อ regime เดียว

ถ้าผ่านทุกข้อ → เริ่ม paper trading 1-3 เดือนก่อน live

► แบบฝึกหัด: คำนวณ **minimum edge** ที่ต้องมีเพื่อ **breakeven** หลัง **cost**

ถ้าเราเทรดบ่อยมาก แต่ edge ต่อ trade น้อย — cost จะกินหมด สูตร minimum edge:

```
def min_edge_to_breakeven( spread_bps: float = 2.0, commission_bps:
float = 7.0, impact_bps: float = 0.7, funding_bps: float = 3.0,
trades_per_year: int = 50, ) -> dict: """ คำนวณ minimum gross alpha ต่อ
trade ที่ strategy ต้องมีเพื่อให้ P&L เป็นบวกหลังหักต้นทุนทั้งหมด """ total_cost_bps =
( spread_bps + # bid-ask (one way) commission_bps + # taker fee round
trip impact_bps + # market impact funding_bps # funding ตลอด holding
period ) # ต้องการ alpha ต่อ trade > total cost min_alpha_bps =
total_cost_bps min_alpha_annual = min_alpha_bps / 10_000 *
trades_per_year print(f"Cost breakdown per trade:") print(f" Spread:
{spread_bps:6.1f} bps") print(f" Commission: {commission_bps:6.1f}
bps") print(f" Impact: {impact_bps:6.1f} bps") print(f" Funding:
{funding_bps:6.1f} bps") print(f" TOTAL COST: {total_cost_bps:6.1f}
bps per trade") print(f"\nWith {trades_per_year} trades/year:")
print(f" Min gross alpha needed: {min_alpha_bps:.1f} bps/trade")
print(f" Min annual gross: {min_alpha_annual:.2%}") return
{ "total_cost_bps": total_cost_bps, "min_alpha_bps": min_alpha_bps,
"min_annual": min_alpha_annual, }
min_edge_to_breakeven( spread_bps=2.0, commission_bps=7.0,
impact_bps=0.7, funding_bps=3.0, trades_per_year=50, )
```

จบ **Part IV** — สิ่งที่ได้จาก **Backtesting**

ก่อน **Part IV**: เทรน model ดูผลดี แต่ไม่รู้ว่า edge จริงหรือแค่ lucky/overfit

หลัง **Part IV**: มีกระบวนการที่เชื่อถือได้ก่อน deploy ทุกครั้ง

- บท 22 — Vectorized backtest เร็วสำหรับ parameter sweep พร้อม `shift(1)` หยุด lookahead bias ✓
- บท 23 — Event-driven backtest สมจริง จำลอง fill และ slippage ทีละ bar ✓
- บท 24 — Walk-forward validation เฝย OOS/IS ratio และ parameter stability ✓
- บท 25 — Realistic cost model (spread + commission + impact + funding) พร้อม sensitivity table ✓

ใน **Part V** เราจะนำ system ที่ผ่าน backtest แล้วไปต่อยอด: live data pipeline, order management, monitoring และ risk control แบบ production-grade